Five-paper research review and implementation feasibility

Group 21, MiniCDN. Prepared 10 October 2026. This is a focused first literature review, not a systematic state-of-the-art survey. The final report template requires at least 12 papers; seven additional studies remain.

Selection and collected data

These five primary sources cover stable placement, capacity-constrained hashing, cache locality and stampede prevention. They provide algorithmic foundations. Their 1997-2021 publication dates do not establish novelty against current commercial products. PDFs were collected for private team review. URLs, DOI, page count and SHA-256 make the sources traceable. Paper-reported datasets/results are distinguished from MiniCDN measurements.

P1: Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web

David Karger, Eric Lehman, Tom Leighton, Matthew Levine, Daniel Lewin, Rina Panigrahy. STOC 1997, pp. 654-663. DOI: 10.1145/258533.258660. Collected PDF: 10 pages.

Finding: Consistent hashing limits reassignment when the cache membership changes. The paper also develops random cache trees to relieve hot spots under explicit theoretical models.

Applicability limit: Stable assignment alone does not split a very popular object's requests across servers. Its modeled guarantees are not a measured latency guarantee for this Go implementation.

What we can do: Use consistent hashing as the locality baseline. Measure request imbalance and key remapping separately from cache hit ratio. Compare it to deterministic round robin on identical traces.

Primary source: Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web.

P2: Consistent Hashing with Bounded Loads

Vahab Mirrokni, Mikkel Thorup, Morteza Zadimoghaddam. SODA 2018, pp. 587-604 (expanded arXiv preprint first uploaded in 2016). DOI: 10.1137/1.9781611975031.39. Collected PDF: 37 pages.

Finding: The allocation algorithm bounds balls per bin by ceil(c*m/n), where c is greater than one. It studies the tradeoff between balance and movement as clients or servers change.

Applicability limit: The bound concerns object/client allocation counts. Requests with unequal popularity, service times, and response sizes require another load model.

What we can do: Study how much locality to retain before redirecting a request. MiniCDN uses in-flight request counts with a slack threshold. This is an experimental heuristic inspired by the tradeoff, not an implementation of this theorem.

Primary source: Consistent Hashing with Bounded Loads.

P3: Revisiting Consistent Hashing with Bounded Loads

John Chen, Benjamin Coleman, Anshumali Shrivastava. AAAI 35(5), pp. 3976-3983. DOI: 10.1609/aaai.v35i5.16517. Collected PDF: 8 pages.

Finding: The authors identify cascaded overflow in capacity-constrained hashing and propose a method using fast minwise hashing. Evaluation includes AOL search and Indiana University click traces.

Applicability limit: The method's object placement and datasets differ from live HTTP request routing. A small synthetic demonstration cannot reproduce its empirical claims.

What we can do: Avoid assuming clockwise overflow always balances load. Record destination distribution and locality loss. Keep minwise object allocation as a separate future baseline rather than labeling our least-loaded fallback as this algorithm.

Primary source: Revisiting Consistent Hashing with Bounded Loads.

P4: Cache-aware load balancing of data center applications

Aaron Archer, Kevin Aydin, MohammadHossein Bateni, Vahab Mirrokni, Aaron Schild, Ray Yang, Richard Zhuang. Proceedings of the VLDB Endowment 12(6), pp. 709-723. DOI: 10.14778/3311880.3311887. Collected PDF: 15 pages.

Finding: The Google search system combines query-term locality with load balancing through weighted voting, graph partitioning, and cache simulation. The production evaluation reports fewer cache misses and increased throughput.

Applicability limit: A multi-term search query uses multiple cached posting lists. MiniCDN currently maps one HTTP resource per request, so directly transplanting the voting scheme would be unjustified.

What we can do: Evaluate cache locality together with request balance. Report origin attempts and latency beside per-edge request counts. A more even distribution may incur additional cold fills.

Primary source: Cache-aware load balancing of data center applications.

P5: Optimal Probabilistic Cache Stampede Prevention

Andrea Vattani, Flavio Chierichetti, Keegan Lowenstein. Proceedings of the VLDB Endowment 8(8), pp. 886-897. DOI: 10.14778/2757807.2757813. Collected PDF: 12 pages.

Finding: The paper analyzes probabilistic early refresh to reduce stampedes around cache expiration, including dependence on regeneration time.

Applicability limit: Early refresh differs from suppressing duplicate simultaneous cache fills. Extra refresh traffic must respect the origin's freshness rules and capacity.

What we can do: Use cache-fill coordination as an implemented baseline. Treat probabilistic early refresh as a later controlled experiment, measuring origin requests, user latency, and wasted refreshes. Do not claim current coordination implements this paper.

Primary source: Optimal Probabilistic Cache Stampede Prevention.

Decision matrix

Candidate	Baseline	MiniCDN evidence	Decision
Cache locality vs traffic imbalance	Round robin vs consistent hash	Same request trace, cache capacity, client timing, per-edge request CV, origin attempts	Implemented; 36 comparison trials recorded
Request-load-aware spillover	Plain hash	In-flight reservations and configurable slack; compare latency vs cold fills	Implemented; latency results mixed
Duplicate cold fills	Coordination off vs on	Barrier-controlled 32-client regression and origin counter	Implemented; established technique, not new invention
Missed invalidation on returning edge	Broadcast only	Retained Redis generation plus heartbeat full flush on mismatch	Implemented; partition recovery passed without restart
Probabilistic early refresh	TTL + coordinated fill	Extra origin requests, expiry-burst latency and wasted refreshes	Defer until core evaluation is reproducible

Research question and falsifiable hypotheses

Question: Under finite cache capacity and skewed/changing demand, can a request-load-aware hash spillover rule reduce tail latency or request imbalance without unacceptable additional origin requests?

H1: Round robin balances request counts better than plain hashing, but may duplicate hot objects across edge caches. H2: Consistent hashing improves locality under some workloads but can concentrate a hot key. H3: Adaptive spillover may trade extra origin fills for lower queueing or imbalance. These are hypotheses; any negative or mixed result is useful and must be reported. A greater-than-85% hit target applies only to an explicitly defined reuse-heavy workload.

The proposed rule preserves the hash owner unless its in-flight count exceeds the least-loaded candidate by more than a slack of two. It does not enforce an origin budget. An origin-budget-aware variant is a future research direction and must not appear as an implemented USP.

Controlled experiment and data schema

Three policies, three repeated runs, four workloads: uniform, Zipf exponent 1.1, changing hot set, and repeated-key cold burst. Each trial starts with a prefix purge acknowledged by all three edges. Use seed 2100260 plus repetition index, 256 objects of 1024 bytes, 64 KiB body capacity per edge, concurrency 16, 768 requests, 20 ms emulated origin delay and one-hour origin freshness. Rotate policy order each run. No warm-up requests are included. This is a closed-loop single-machine Docker experiment.

Record individual latency, HTTP status, key, selected edge and cache status. Derive P50/P95/P99 from raw client measurements, successful throughput, response-body validation errors, request CV across all three edges, cache HIT fraction and origin/coalescing counter deltas. Save JSON, CSV and raw JSONL. Counters are sampled directly at each edge before/after each trial; Grafana provides live observability. Report each repeat and mean/range across repeats, not only the best run.

Caveats: equal body capacity excludes metadata/temporary buffers; equal request counts do not equal CPU load. Closed-loop traffic can hide overload behavior. Origin delay is synthetic, node regions are local labels, and CRC32 virtual-node placement is measured rather than assumed uniform. Cache coalescing changes apparent HIT status of an initial miss, so HIT fraction and actual origin attempts must both be reported. Test failure recovery separately from the no-failure comparison.

Proposed project positioning

MiniCDN is a transparent CDN experiment platform with interchangeable routing policies, observable cache fills, finite storage, invalidation recovery and reproducible comparisons. This is a defensible educational/project value proposition. Five reviewed papers cannot prove that Cloudflare or every modern solution lacks a feature. A scientific novelty claim would require broader literature, commercial documentation review and a clearly superior evaluated mechanism.

Reading and final report expansion

Add at least seven relevant papers before ESE: modern cache eviction/admission, distributed membership/failure detection, invalidation consistency, cooperative caching, and realistic trace-based evaluation. Record exact sources and assumptions. Do not use unrelated template bibliography placeholders. A research-paper manuscript can describe measured results after they exist; publication acceptance and plagiarism evidence cannot be generated as facts.